

1. OBJECTIF DU PROJET

Application JavaFX d'administration d'utilisateurs avec authentification admin renforcee (CAPTCHA, 2FA email, FaceID), gestion CRUD et alertes de securite.

2. STACK TECHNIQUE

- Java 17 + JavaFX 17
- Maven (build + execution JavaFX)
- MySQL via JDBC (mysql-connector-j)
- Jakarta Mail (emails OTP/alertes)
- Webcam Capture (capture image/FaceID)

3. STRUCTURE DU PROJET

Guser/

.mvn/

.idea/

src/

main/

java/

com/guser/

MainApp.java

config/DatabaseConfig.java

config/DatabaseConnection.java

config/MailConfig.java

controller/DashboardController.java

controller/LoginController.java

controller/UserController.java

dao/UserDao.java

model/User.java

service/CaptchaApiService.java

service/EmailService.java

service/FaceIdService.java

util/SceneManager.java

util/SessionManager.java

resources/

database.sql

com/guser/

login-view.fxml

user-view.fxml

dashboard-view.fxml

app.css

assets/

logo-otemps.png

logo.png

pom.xml

target/ (artefacts Maven compiles)

4. INVENTAIRE GLOBAL DES CLASSES ET METHODES

- com.guser.config.DatabaseConfig: 4 methodes/constructeurs
- com.guser.config.DatabaseConnection: 2 methodes/constructeurs

- com.guser.config.MailConfig: 6 methodes/constructeurs
- com.guser.controller.DashboardController: 4 methodes/constructeurs
- com.guser.controller.LoginController: 17 methodes/constructeurs
- com.guser.controller.UserController: 21 methodes/constructeurs
- com.guser.dao.UserDao: 20 methodes/constructeurs
- com.guser.MainApp: 3 methodes/constructeurs
- com.guser.model.User: 25 methodes/constructeurs
- com.guser.service.CaptchaApiService: 3 methodes/constructeurs
- com.guser.service.EmailService: 14 methodes/constructeurs
- com.guser.service.FaceldService: 9 methodes/constructeurs
- com.guser.util.SceneManager: 2 methodes/constructeurs
- com.guser.util.SessionManager: 4 methodes/constructeurs

TOTAL METHODES/CONSTRUCTEURS DETECTES: 134

5. DETAIL COMPLET PAR CLASSE

=====

Classe: com.guser.config.DatabaseConfig

Fichier: src/main/java/com/guser/config/DatabaseConfig.java

Responsabilite: Centralise les parametres de connexion MySQL (URL, utilisateur, mot de passe) via des accesseurs statiques.

Dependances principales: Aucune importation explicite.

Nombre de methodes/constructeurs: 4

1. Signature: private DatabaseConfig()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public static String getUrl()

Role: Expose la valeur courante de 'url' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public static String getUsername()

Role: Expose la valeur courante de 'username' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: public static String getPassword()

Role: Expose la valeur courante de 'password' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====

Classe: com.guser.config.DatabaseConnection

Fichier: src/main/java/com/guser/config/DatabaseConnection.java

Responsabilite: Expose une fabrique de connexion JDBC en s'appuyant sur DatabaseConfig.

Page 2/20

Dependances principales: java.sql.Connection, java.sql.DriverManager, java.sql.SQLException
Nombre de methodes/constructeurs: 2

1. Signature: private DatabaseConnection()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public static Connection getConnection() throws SQLException

Role: Expose la valeur courante de 'connection' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: Connection.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====
Classe: com.guser.config.MailConfig

Fichier: src/main/java/com/guser/config/MailConfig.java

Responsabilite: Centralise la configuration SMTP utilisee par EmailService.

Dependances principales: Aucune importation explicite.

Nombre de methodes/constructeurs: 6

1. Signature: private MailConfig()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public static String getSmtphost()

Role: Expose la valeur courante de 'smtp host' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public static int getSmtpport()

Role: Expose la valeur courante de 'smtp port' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: int.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: public static String getUsername()

Role: Expose la valeur courante de 'username' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

5. Signature: public static String getPassword()

Role: Expose la valeur courante de 'password' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

6. Signature: public static String getFromAddress()

Role: Expose la valeur courante de 'from address' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====
Classe: com.guser.controller.DashboardController

Fichier: src/main/java/com/guser/controller/DashboardController.java

Responsabilite: Controleur JavaFX du tableau de bord admin (metriques, navigation vers les utilisateurs, deconnexion).

Dependances principales: com.guser.dao.UserDao, com.guser.model.User, com.guser.util.SceneManager, com.guser.util.SessionManager, javafx.fxml.FXML, javafx.scene.control.Alert, javafx.scene.control.Label, java.sql.SQLException

Nombre de methodes/constructeurs: 4

1. Signature: public void initialize()

Role: Alimente l'entete avec l'utilisateur de session puis recharge les metriques du dashboard.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: private void ouvrirUtilisateurs()

Role: Ouvre la vue de gestion des utilisateurs.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: private void deconnexion()

Role: Vide la session en memoire puis renvoie vers l'ecran de connexion.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: private void refreshMetrics()

Role: Calcule le total des utilisateurs via UserDao et met a jour les labels d'indicateurs.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL; bloque l'UI pendant l'attente utilisateur.

=====
Classe: com.guser.controller.LoginController

Fichier: src/main/java/com/guser/controller/LoginController.java

Responsabilite: Controleur JavaFX de l'authentification : captcha, mot de passe, 2FA email, oubli de mot de passe et login FaceID.

Dependances principales: com.github.sarxos.webcam.Webcam, com.guser.dao.UserDao, com.guser.model.User, com.guser.service.CaptchaApiService, com.guser.service.EmailService, com.guser.service.FaceIdService, com.guser.util.SceneManager, com.guser.util.SessionManager

Nombre de methodes/constructeurs: 17

1. Signature: private void initialize()

Role: Implante la logique de 'initialize' selon le contexte de la classe LoginController.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: private void seConnecter()

Role: Flux principal d'authentification admin: validations de saisie, captcha, mot de passe, role ADMIN, puis 2FA email.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL; peut echouer si SMTP est mal configure.

3. Signature: private void ouvrirMonTravail()

Role: Gere la logique metier de 'ouvrir mon travail' avec les operations suivantes: navigation entre vues JavaFX, feedback u

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: private void seConnecterParVisage()

Role: Authentifie un admin par comparaison FaceID entre template capture et templates en base.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

5. Signature: private void motDePasseOublie()

Role: Declenche la recuperation par email (code OTP) puis met a jour le mot de passe en base.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL; peut echouer si SMTP est mal configure.

6. Signature: private boolean askCode(String expectedCode)

Role: Implante la logique de 'ask code' selon le contexte de la classe LoginController.

Parametres: String expectedCode

Retour: Retourne: boolean.

Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

7. Signature: private Optional<String> askNewPassword()

Role: Gere la logique metier de 'ask new password' avec les operations suivantes: validation de format, retour optionnel, fee

Parametres: Aucun parametre.

Retour: Retourne: Optional<String>.

Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

8. Signature: private String generateCode()

Role: Gere la logique metier de 'generate code' avec les operations suivantes: generation pseudo-aleatoire.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

9. Signature: private void regenererCaptcha()

Role: Implante la logique de 'regenerer captcha' selon le contexte de la classe LoginController.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

10. Signature: private void refreshCaptcha()
Role: Implante la logique de 'refresh captcha' selon le contexte de la classe LoginController.
Parametres: Aucun parametre.
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
11. Signature: private boolean validateTwoFactor(User user) throws MessagingException
Role: Genere et envoie un code 2FA temporaire, puis delegate la verification a une boite de dialogue.
Parametres: User user
Retour: Retourne: boolean.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
12. Signature: private boolean askTwoFactorCode(String expectedCode, LocalDateTime expiresAt)
Role: Gere la logique metier de 'ask two factor code' avec les operations suivantes: feedback utilisateur.
Parametres: String expectedCode, LocalDateTime expiresAt
Retour: Retourne: boolean.
Risques/contraintes: bloque l'UI pendant l'attente utilisateur.
13. Signature: private void handleFailedPasswordAttempt(String email) throws SQLException
Role: Compte les echecs de connexion par email et declenche une capture camera au 3e echec.
Parametres: String email
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
14. Signature: private void resetFailedAttempts(String email)
Role: Implante la logique de 'reset failed attempts' selon le contexte de la classe LoginController.
Parametres: String email
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
15. Signature: private void sendCaptureToUser(User user)
Role: Capture une image webcam et l'envoie en piece jointe d'un email d'alerte de securite.
Parametres: User user
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: peut echouer si SMTP est mal configure.
16. Signature: private Path captureFromCamera() throws IOException
Role: Gere la logique metier de 'capture from camera' avec les operations suivantes: interaction camera.
Parametres: Aucun parametre.
Retour: Retourne: Path.
Risques/contraintes: necessite une camera accessible.
17. Signature: private void showAlert(Alert.AlertType type, String message)
Role: Gere la logique metier de 'show alert' avec les operations suivantes: feedback utilisateur.
Parametres: Alert.AlertType type, String message
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

=====
Classe: com.guser.controller.UserController

Fichier: src/main/java/com/guser/controller/UserController.java

Responsabilite: Controleur JavaFX de gestion des utilisateurs : CRUD, recherche, tri, validation

des champs et enregistrement FaceID.

Dependances principales: com.guser.dao.UserDao, com.guser.model.User, com.guser.service.EmailService, com.guser.service.FaceIdService, com.guser.util.SceneManager, com.guser.util.SessionManager, jakarta.mail.MessagingException, javafx.application.Platform

Nombre de methodes/constructeurs: 21

1. Signature: public void initialize()

Role: Controle l'accès ADMIN, initialise la liste des rôles et les listeners UI, puis charge les utilisateurs.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: private void ajouterUtilisateur()

Role: Valide les champs, envoie un code de verification email, puis insere l'utilisateur si le code est confirme.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL; peut echouer si SMTP est mal configure.

3. Signature: private void modifierUtilisateur()

Role: Met a jour l'utilisateur selectionne apres validation metier et persiste les changements en base.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

4. Signature: private void supprimerUtilisateur()

Role: Supprime l'utilisateur selectionne apres confirmation explicite.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL; bloque l'UI pendant l'attente utilisateur.

5. Signature: private void trierParNom()

Role: Implante la logique de 'trier par nom' selon le contexte de la classe UserController.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

6. Signature: private void deconnexion()

Role: Gere la logique metier de 'deconnexion' avec les operations suivantes: navigation entre vues JavaFX.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

7. Signature: private void enregistrerFaceId()

Role: Capture un template FaceID et le lie a l'utilisateur ADMIN selectionne.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

8. Signature: private void supprimerFaceId()

Role: Implante la logique de 'supprimer face id' selon le contexte de la classe UserController.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

9. Signature: private void refreshUsers()

Role: Implante la logique de 'refresh users' selon le contexte de la classe UserController.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

10. Signature: private void renderUserCards()

Role: Reconstitue la grille des cartes selon recherche et ordre de tri.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

11. Signature: private VBox createUserCard(User user)

Role: Implante la logique de 'create user card' selon le contexte de la classe UserController.

Parametres: User user

Retour: Retourne: VBox.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

12. Signature: private Region spacer()

Role: Implante la logique de 'spacer' selon le contexte de la classe UserController.

Parametres: Aucun parametre.

Retour: Retourne: Region.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

13. Signature: private Label createMetaLabel(String text)

Role: Implante la logique de 'create meta label' selon le contexte de la classe UserController.

Parametres: String text

Retour: Retourne: Label.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

14. Signature: private String formatCreationDate(User user)

Role: Implante la logique de 'format creation date' selon le contexte de la classe UserController.

Parametres: User user

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

15. Signature: private void selectUser(User user)

Role: Implante la logique de 'select user' selon le contexte de la classe UserController.

Parametres: User user

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

16. Signature: private boolean validateFields(User editingUser)

Role: Applique les regles metier sur nom/prenoms/email/mot de passe et unicite email.

Parametres: User editingUser

Retour: Retourne: boolean.

Risques/contraintes: depend de la disponibilite MySQL.

17. Signature: private boolean demanderValidationCode(String expectedCode)

Role: Implante la logique de 'demander validation code' selon le contexte de la classe UserController.

Parametres: String expectedCode
Retour: Retourne: boolean.
Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

18. Signature: private String generateVerificationCode()

Role: Gere la logique metier de 'generate verification code' avec les operations suivantes: generation pseudo-aleatoire.
Parametres: Aucun parametre.
Retour: Retourne: String.
Risques/contraintes: code pseudo-aleatoire non cryptographique.

19. Signature: private void clearFields()

Role: Implante la logique de 'clear fields' selon le contexte de la classe UserController.
Parametres: Aucun parametre.
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

20. Signature: private void showInfo(String message)

Role: Gere la logique metier de 'show info' avec les operations suivantes: feedback utilisateur.
Parametres: String message
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

21. Signature: private void showError(String message)

Role: Gere la logique metier de 'show error' avec les operations suivantes: feedback utilisateur.
Parametres: String message
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

=====

Classe: com.guser.dao.UserDao

Fichier: src/main/java/com/guser/dao/UserDao.java

Responsabilite: Couche d'accès aux données utilisateurs (CRUD, auth, verifications, evolution automatique du schema).

Dependances principales: com.guser.config.DatabaseConnection, com.guser.model.User, java.sql.Connection, java.sql.DatabaseMetaData, java.sql.PreparedStatement, java.sql.ResultSet, java.sql.SQLException, java.sql.Statement

Nombre de methodes/constructeurs: 20

1. Signature: public UserDao()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.
Parametres: Aucun parametre.
Retour: Constructeur (initialise une instance).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public List<User> findAll() throws SQLException

Role: Retourne tous les utilisateurs tries du plus recent au plus ancien.
Parametres: Aucun parametre.
Retour: Retourne: List<User>.
Risques/contraintes: depend de la disponibilite MySQL.

3. Signature: public Optional<User> authenticate(String email, String motDePasse) throws SQLException

Role: Verifie le couple email/mot_de_passe et retourne eventuellement l'utilisateur.

Parametres: String email, String motDePasse

Retour: Retourne: Optional<User>.

Risques/contraintes: depend de la disponibilite MySQL.

4. Signature: public void insert(User user) throws SQLException

Role: Gere la logique metier de 'insert' avec les operations suivantes: ecriture SQL.

Parametres: User user

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

5. Signature: public void update(User user) throws SQLException

Role: Gere la logique metier de 'update' avec les operations suivantes: ecriture SQL.

Parametres: User user

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

6. Signature: public void delete(int userId) throws SQLException

Role: Gere la logique metier de 'delete' avec les operations suivantes: ecriture SQL.

Parametres: int userId

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

7. Signature: public boolean existsByEmail(String email) throws SQLException

Role: Gere la logique metier de 'exists by email' avec les operations suivantes: lecture SQL.

Parametres: String email

Retour: Retourne: boolean.

Risques/contraintes: depend de la disponibilite MySQL.

8. Signature: public boolean existsByEmailExceptId(String email, int userId) throws SQLException

Role: Gere la logique metier de 'exists by email except id' avec les operations suivantes: lecture SQL.

Parametres: String email, int userId

Retour: Retourne: boolean.

Risques/contraintes: depend de la disponibilite MySQL.

9. Signature: public Optional<User> findByEmail(String email) throws SQLException

Role: Gere la logique metier de 'find by email' avec les operations suivantes: lecture SQL, retour optionnel.

Parametres: String email

Retour: Retourne: Optional<User>.

Risques/contraintes: depend de la disponibilite MySQL.

10. Signature: public void updatePassword(int userId, String newPassword) throws SQLException

Role: Gere la logique metier de 'update password' avec les operations suivantes: ecriture SQL.

Parametres: int userId, String newPassword

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: depend de la disponibilite MySQL.

11. Signature: public List<User> findAdminsWithFaceTemplate() throws SQLException

Role: Retourne les comptes ADMIN qui possedent deja un template FaceID exploitable.

Parametres: Aucun parametre.

Retour: Retourne: List<User>.

Risques/contraintes: depend de la disponibilite MySQL.

12. Signature: `public void updateFaceTemplate(int userId, String faceTemplate) throws SQLException`
Role: Gere la logique metier de 'update face template' avec les operations suivantes: ecriture SQL.
Parametres: `int userId`, `String faceTemplate`
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: depend de la disponibilite MySQL.
13. Signature: `public void clearFaceTemplate(int userId) throws SQLException`
Role: Implante la logique de 'clear face template' selon le contexte de la classe UserDao.
Parametres: `int userId`
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
14. Signature: `public int countAll() throws SQLException`
Role: Gere la logique metier de 'count all' avec les operations suivantes: lecture SQL.
Parametres: Aucun parametre.
Retour: Retourne: `int`.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
15. Signature: `private void ensureInitialized() throws SQLException`
Role: Implante la logique de 'ensure initialized' selon le contexte de la classe UserDao.
Parametres: Aucun parametre.
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
16. Signature: `private void ensureSchema() throws SQLException`
Role: Cree/ajuste automatiquement la table utilisateurs pour maintenir la compatibilite du schema.
Parametres: Aucun parametre.
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
17. Signature: `private boolean hasColumn(DatabaseMetaData metaData, String columnName) throws SQLException`
Role: Implante la logique de 'has column' selon le contexte de la classe UserDao.
Parametres: `DatabaseMetaData metaData`, `String columnName`
Retour: Retourne: `boolean`.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
18. Signature: `private User mapUser(ResultSet resultSet) throws SQLException`
Role: Implante la logique de 'map user' selon le contexte de la classe UserDao.
Parametres: `ResultSet resultSet`
Retour: Retourne: `User`.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
19. Signature: `private java.time.LocalDateTime toLocalDateTime(Timestamp timestamp)`
Role: Implante la logique de 'to local date time' selon le contexte de la classe UserDao.
Parametres: `Timestamp timestamp`
Retour: Retourne: `java.time.LocalDateTime`.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.
20. Signature: `private int countWithQuery(String sql) throws SQLException`
Role: Gere la logique metier de 'count with query' avec les operations suivantes: lecture SQL.

Parametres: String sql

Retour: Retourne: int.

Risques/contraintes: depend de la disponibilite MySQL.

=====

Classe: com.guser.MainApp

Fichier: src/main/java/com/guser/MainApp.java

Responsabilite: Point d'entree JavaFX. Charge la vue de connexion, applique le style global et gere un mode de secours si le chargement echoue.

Dependances principales: javafx.application.Application, javafx.fxml.FXMLLoader, javafx.scene.Scene, javafx.scene.control.Alert, javafx.scene.control.TextArea, javafx.scene.layout.VBox, javafx.stage.Stage

Nombre de methodes/constructeurs: 3

1. Signature: public void start(Stage stage)

Role: Initialise la fenetre principale, charge login-view.fxml et applique app.css; bascule sur une vue d'erreur si un probleme

Parametres: Stage stage

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: bloque l'UI pendant l'attente utilisateur.

2. Signature: private String buildErrorMessage(Exception exception)

Role: Construit une chaine d'erreur lisible en parcourant la chaine des causes de l'exception.

Parametres: Exception exception

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public static void main(String[] args)

Role: Lance le cycle de vie JavaFX via Application.launch.

Parametres: String[] args

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====

Classe: com.guser.model.User

Fichier: src/main/java/com/guser/model/User.java

Responsabilite: Modele metier utilisateur (etat, constructeurs, getters/setters et helpers).

Dependances principales: java.time.LocalDateTime

Nombre de methodes/constructeurs: 25

1. Signature: public User()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public User(int id, String nom, String prenom, String email, String telephone, String motDePasse, String role, LocalDateTime dateCreation)

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: int id, String nom, String prenom, String email, String telephone, String motDePasse, String role, LocalDateTime dateCreation

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: `public User(int id, String nom, String prenom, String email, String telephone, String motDePasse, String role, LocalDateTime dateCreation, String faceTemplate)`
Role: Construit l'objet et positionne les attributs initiaux nécessaires à son fonctionnement.
Paramètres: `int id, String nom, String prenom, String email, String telephone, String motDePasse, String role, LocalDateTime dateCreation, String faceTemplate`
Retour: Constructeur (initialise une instance).
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
4. Signature: `public User(String nom, String prenom, String email, String telephone, String motDePasse, String role)`
Role: Construit l'objet et positionne les attributs initiaux nécessaires à son fonctionnement.
Paramètres: `String nom, String prenom, String email, String telephone, String motDePasse, String role`
Retour: Constructeur (initialise une instance).
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
5. Signature: `public User(String nom, String prenom, String email, String telephone, String motDePasse, String role, String faceTemplate)`
Role: Construit l'objet et positionne les attributs initiaux nécessaires à son fonctionnement.
Paramètres: `String nom, String prenom, String email, String telephone, String motDePasse, String role, String faceTemplate`
Retour: Constructeur (initialise une instance).
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
6. Signature: `public int getId()`
Role: Expose la valeur courante de 'id' sans modifier l'état interne.
Paramètres: Aucun paramètre.
Retour: Retourne: `int`.
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
7. Signature: `public void setId(int id)`
Role: Met à jour l'attribut 'id' avec la valeur fournie en paramètre.
Paramètres: `int id`
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
8. Signature: `public String getNom()`
Role: Expose la valeur courante de 'nom' sans modifier l'état interne.
Paramètres: Aucun paramètre.
Retour: Retourne: `String`.
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
9. Signature: `public void setNom(String nom)`
Role: Met à jour l'attribut 'nom' avec la valeur fournie en paramètre.
Paramètres: `String nom`
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
10. Signature: `public String getPrenoms()`
Role: Expose la valeur courante de 'prenom' sans modifier l'état interne.
Paramètres: Aucun paramètre.
Retour: Retourne: `String`.
Risques/contraintes: Risque faible; méthode locale sans dépendance externe critique.
11. Signature: `public void setPrenoms(String prenom)`

Role: Met a jour l'attribut 'prenoms' avec la valeur fournie en parametre.

Parametres: String prenoms

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

12. Signature: public String getEmail()

Role: Expose la valeur courante de 'email' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

13. Signature: public void setEmail(String email)

Role: Met a jour l'attribut 'email' avec la valeur fournie en parametre.

Parametres: String email

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

14. Signature: public String getTelephone()

Role: Expose la valeur courante de 'telephone' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

15. Signature: public void setTelephone(String telephone)

Role: Met a jour l'attribut 'telephone' avec la valeur fournie en parametre.

Parametres: String telephone

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

16. Signature: public String getMotDePasse()

Role: Expose la valeur courante de 'mot de passe' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

17. Signature: public void setMotDePasse(String motDePasse)

Role: Met a jour l'attribut 'mot de passe' avec la valeur fournie en parametre.

Parametres: String motDePasse

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

18. Signature: public String getRole()

Role: Expose la valeur courante de 'role' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

19. Signature: public void setRole(String role)

Role: Met a jour l'attribut 'role' avec la valeur fournie en parametre.

Parametres: String role

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

20. Signature: public LocalDateTime getDateCreation()
Role: Expose la valeur courante de 'date creation' sans modifier l'etat interne.
Parametres: Aucun parametre.
Retour: Retourne: LocalDateTime.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

21. Signature: public void setDateCreation(LocalDateTime dateCreation)
Role: Met a jour l'attribut 'date creation' avec la valeur fournie en parametre.
Parametres: LocalDateTime dateCreation
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

22. Signature: public String getFaceTemplate()
Role: Expose la valeur courante de 'face template' sans modifier l'etat interne.
Parametres: Aucun parametre.
Retour: Retourne: String.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

23. Signature: public void setFaceTemplate(String faceTemplate)
Role: Met a jour l'attribut 'face template' avec la valeur fournie en parametre.
Parametres: String faceTemplate
Retour: Aucune valeur retour (effet de bord).
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

24. Signature: public boolean hasFaceTemplate()
Role: Effectue une verification booleenne sur l'etat de l'objet.
Parametres: Aucun parametre.
Retour: Retourne: boolean.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

25. Signature: public String getNomComplet()
Role: Expose la valeur courante de 'nom complet' sans modifier l'etat interne.
Parametres: Aucun parametre.
Retour: Retourne: String.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====
Classe: com.guser.service.CaptchaApiService

Fichier: src/main/java/com/guser/service/CaptchaApiService.java

Responsabilite: Genere un code captcha et tente de recuperer son image depuis un service HTTP externe.

Dependances principales: javafx.scene.image.Image, java.io.InputStream, java.net.URI, java.net.URLEncoder, java.nio.charset.StandardCharsets, java.util.Random

Nombre de methodes/constructeurs: 3

1. Signature: public CaptchaChallenge generateChallenge()
Role: Implante la logique de 'generate challenge' selon le contexte de la classe CaptchaApiService.
Parametres: Aucun parametre.
Retour: Retourne: CaptchaChallenge.
Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: private String generateCode()

Role: Gere la logique metier de 'generate code' avec les operations suivantes: generation pseudo-aleatoire.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: private Image loadCaptchalImage(String code)

Role: Implante la logique de 'load captcha image' selon le contexte de la classe CaptchaApiService.

Parametres: String code

Retour: Retourne: Image.

Risques/contraintes: depend d'un service HTTP externe.

=====
Classe: com.guser.service.EmailService

Fichier: src/main/java/com/guser/service/EmailService.java

Responsabilite: Construit et envoie des emails HTML brandes (verification, reset, 2FA, alerte de securite + piece jointe).

Dependances principales: com.guser.config.MailConfig, com.guser.dao.UserDao, com.guser.model.User, jakarta.activation.DataHandler, jakarta.mail.Authenticator, jakarta.mail.Message, jakarta.mail.MessagingException, jakarta.mail.PasswordAuthentication

Nombre de methodes/constructeurs: 14

1. Signature: public void sendVerificationCode(String recipient, String fullName, String code)

throws MessagingException

Role: Implante la logique de 'send verification code' selon le contexte de la classe EmailService.

Parametres: String recipient, String fullName, String code

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public void sendPasswordResetCode(String recipient, String fullName, String code)

throws MessagingException

Role: Implante la logique de 'send password reset code' selon le contexte de la classe EmailService.

Parametres: String recipient, String fullName, String code

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public void sendTwoFactorCode(String recipient, String fullName, String code,

LocalDateTime expiresAt) throws MessagingException

Role: Implante la logique de 'send two factor code' selon le contexte de la classe EmailService.

Parametres: String recipient, String fullName, String code, LocalDateTime expiresAt

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: public void sendSuspiciousLoginCapture(String recipient, String fullName, File

imageFile) throws MessagingException, IOException

Role: Gere la logique metier de 'send suspicious login capture' avec les operations suivantes: interaction camera, feedback

Parametres: String recipient, String fullName, File imageFile

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

5. Signature: private Session createSession()

Role: Implante la logique de 'create session' selon le contexte de la classe EmailService.

Parametres: Aucun parametre.

Retour: Retourne: Session.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

6. Signature: private void sendBrandedEmail(String recipient, String fallbackName, String subject, String title, String intro, String codeLabel, String codeValue, String footerText, File attachment, String attachmentFileName) throws MessagingException

Role: Assemble un email HTML complet (logo inline, contenu, eventuelle piece jointe) puis l'envoi via SMTP.

Parametres: String recipient, String fallbackName, String subject, String title, String intro, String codeLabel, String codeValue

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: peut echouer si SMTP est mal configure.

7. Signature: private MimeBodyPart createInlineLogoPart(byte[] logoBytes) throws MessagingException

Role: Gere la logique metier de 'create inline logo part' avec les operations suivantes: envoi d'email.

Parametres: byte[] logoBytes

Retour: Retourne: MimeBodyPart.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

8. Signature: private byte[] loadLogoBytes()

Role: Implante la logique de 'load logo bytes' selon le contexte de la classe EmailService.

Parametres: Aucun parametre.

Retour: Retourne: byte[].

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

9. Signature: private byte[] loadResourceBytes(String resourcePath)

Role: Implante la logique de 'load resource bytes' selon le contexte de la classe EmailService.

Parametres: String resourcePath

Retour: Retourne: byte[].

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

10. Signature: private String resolveRecipientName(String recipient, String fallbackName)

Role: Determine un nom destinataire via base de donnees, fallback fourni, ou derive de l'email.

Parametres: String recipient, String fallbackName

Retour: Retourne: String.

Risques/contraintes: depend de la disponibilite MySQL.

11. Signature: private String capitalize(String input)

Role: Implante la logique de 'capitalize' selon le contexte de la classe EmailService.

Parametres: String input

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

12. Signature: private String buildEmailHtml(String recipientName, String title, String intro, String codeLabel, String codeValue, String footerText, boolean includeLogo)

Role: Implante la logique de 'build email html' selon le contexte de la classe EmailService.

Parametres: String recipientName, String title, String intro, String codeLabel, String codeValue, String footerText, boolean includeLogo

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

13. Signature: private String escapeHtml(String text)

Role: Implante la logique de 'escape html' selon le contexte de la classe EmailService.

Parametres: String text

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

14. Signature: private Properties buildProperties()

Role: Gere la logique metier de 'build properties' avec les operations suivantes: envoi d'email.

Parametres: Aucun parametre.

Retour: Retourne: Properties.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====

Classe: com.guser.service.FaceIdService

Fichier: src/main/java/com/guser/service/FaceIdService.java

Responsabilite: Capture webcam et calcule un template biometrie (LBP), puis compare des templates par similarite cosinus.

Dependances principales: com.github.sarxos.webcam.Webcam, java.awt.Graphics2D, java.awt.RenderingHints, java.awt.image.BufferedImage, java.awt.image.Raster, java.io.IOException, java.nio.ByteBuffer, java.util.Base64

Nombre de methodes/constructeurs: 9

1. Signature: public String captureTemplate() throws IOException

Role: Pipeline biometrie complet: capture webcam, recadrage, grayscale, histogramme LBP et encodage Base64.

Parametres: Aucun parametre.

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public double compareTemplates(String liveTemplate, String enrolledTemplate)

Role: Calcule une similarite cosinus entre deux templates FaceID pour produire un score [0..1].

Parametres: String liveTemplate, String enrolledTemplate

Retour: Retourne: double.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public double getMatchThreshold()

Role: Expose la valeur courante de 'match threshold' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: double.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: private BufferedImage captureFromCamera() throws IOException

Role: Gere la logique metier de 'capture from camera' avec les operations suivantes: interaction camera.

Parametres: Aucun parametre.

Retour: Retourne: BufferedImage.

Risques/contraintes: necessite une camera accessible.

5. Signature: private BufferedImage cropCenterSquare(BufferedImage image)

Role: Implante la logique de 'crop center square' selon le contexte de la classe FaceIdService.

Parametres: BufferedImage image

Retour: Retourne: BufferedImage.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

6. Signature: private BufferedImage toGrayscale(BufferedImage image, int width, int height)

Role: Implante la logique de 'to grayscale' selon le contexte de la classe FaceIdService.

Parametres: BufferedImage image, int width, int height

Retour: Retourne: BufferedImage.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

7. Signature: private float[] buildLbpHistogram(BufferedImage image)

Role: Implante la logique de 'build lbp histogram' selon le contexte de la classe FaceldService.

Parametres: BufferedImage image

Retour: Retourne: float[].

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

8. Signature: private String encodeHistogram(float[] histogram)

Role: Gere la logique metier de 'encode histogram' avec les operations suivantes: encodage/decodage binaire.

Parametres: float[] histogram

Retour: Retourne: String.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

9. Signature: private float[] decodeHistogram(String encodedHistogram)

Role: Gere la logique metier de 'decode histogram' avec les operations suivantes: encodage/decodage binaire.

Parametres: String encodedHistogram

Retour: Retourne: float[].

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====
Classe: com.guser.util.SceneManager

Fichier: src/main/java/com/guser/util/SceneManager.java

Responsabilite: Utilitaire de navigation JavaFX pour changer de vue/FXML et reappliquer la feuille CSS.

Dependances principales: com.guser.MainApp, javafx.fxml.FXMLLoader, javafx.scene.Node, javafx.scene.Parent, javafx.scene.Scene, javafx.stage.Stage, java.io.IOException

Nombre de methodes/constructeurs: 2

1. Signature: private SceneManager()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public static void switchScene(Node node, String fxmlPath, String title)

Role: Implante la logique de 'switch scene' selon le contexte de la classe SceneManager.

Parametres: Node node, String fxmlPath, String title

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====
Classe: com.guser.util.SessionManager

Fichier: src/main/java/com/guser/util/SessionManager.java

Responsabilite: Stockage en memoire de l'utilisateur connecte pour la session applicative courante.

Dependances principales: com.guser.model.User

Nombre de methodes/constructeurs: 4

1. Signature: private SessionManager()

Role: Construit l'objet et positionne les attributs initiaux necessaires a son fonctionnement.

Parametres: Aucun parametre.

Retour: Constructeur (initialise une instance).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

2. Signature: public static void setCurrentUser(User user)

Role: Met a jour l'attribut 'current user' avec la valeur fournie en parametre.

Parametres: User user

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

3. Signature: public static User getCurrentUser()

Role: Expose la valeur courante de 'current user' sans modifier l'etat interne.

Parametres: Aucun parametre.

Retour: Retourne: User.

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

4. Signature: public static void clear()

Role: Implante la logique de 'clear' selon le contexte de la classe SessionManager.

Parametres: Aucun parametre.

Retour: Aucune valeur retour (effet de bord).

Risques/contraintes: Risque faible; methode locale sans dependance externe critique.

=====

6. OBSERVATIONS TECHNIQUES IMPORTANTES

- Les mots de passe utilisateurs sont compares/stockes en clair dans la logique actuelle (pas de hachage).
- Les identifiants SMTP et JDBC sont codes en dur dans les classes de configuration.
- FaceID est une implementation locale LBP/cosinus: utile pour prototype, a durcir pour production.
- UserDao applique une migration automatique de schema au demarrage (avantage: resiliente; risque: operation non tracee).

FIN DU DOCUMENT

Page 20/20